DBMS

University Textbook

Table of Contents

Introduction to Database Management Systems

Chapter 1 — Fundamentals of DBMS

A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to a database. It serves as an interface between the database and its users or applications, ensuring that data is consistently organized and remains easily accessible.

1.1 What is a Database?

Definition: A database is an organized collection of structured data stored electronically, designed to allow efficient retrieval, insertion, updating, and deletion of data.

Databases are used in virtually every domain — banking, e-commerce, healthcare, education, and more. The core objective is to eliminate data redundancy, maintain data integrity, and enable multi-user access.

1.2 Advantages of DBMS over File System

- Eliminates data redundancy and inconsistency
- Provides data sharing among multiple users
- Enforces data integrity through constraints
- Supports data security and access control
- Enables data abstraction and independence
- Supports concurrent access and transaction management

1.3 Types of Database Models

Model	Description	Example
Hierarchical	Data organized in tree structure with parent-child relationships	IBM IMS
Network	Data organized as a graph allowing multiple parent nodes	IDS
Relational	Data stored in tables (relations) with rows and columns	MySQL, PostgreSQL
Object-Oriented	Data stored as objects similar to OOP concepts	ObjectDB
NoSQL	Non-tabular storage for unstructured or semi-structured data	MongoDB, Cassandra

1.4 Three-Schema Architecture

The ANSI/SPARC three-schema architecture separates the database into three levels to achieve data abstraction and independence:

- External Schema (View Level): Describes how individual users see the data. Different users can have different views.
- Conceptual Schema (Logical Level): Describes the overall logical structure — all entities, relationships, and constraints.
- Internal Schema (Physical Level): Describes how data is physically stored on disk — file organization, indexing, and storage.

Note: Data independence means changes at one level do not affect the level above it. Logical data independence shields the external level from conceptual changes; physical data independence shields the conceptual level from storage changes.

1.5 Roles in a Database Environment

Role	Responsibilities
Database Administrator (DBA)	Manages schema, access control, backup, recovery, and performance tu
Database Designer	Designs the logical and physical schema of the database
Application Developer	Writes programs that interact with the database via SQL or APIs
End User	Queries and updates the database through applications

1.6 Summary

A DBMS provides a reliable and efficient way to store and manage large volumes of data. Understanding the three-schema architecture and data models is the foundation for learning advanced concepts such as relational algebra, normalization, and transaction management covered in subsequent chapters.

Introduction to Database Management Systems

Chapter 2 — Relational Model & SQL

The relational model, introduced by E.F. Codd in 1970, organizes data into relations (tables) consisting of tuples (rows) and attributes (columns). It remains the most widely used database model today, forming the foundation of systems like MySQL, PostgreSQL, Oracle, and SQL Server.

2.1 Key Terminology

Term	Meaning
Relation (Table)	A two-dimensional structure with rows and columns
Tuple (Row)	A single record in a relation
Attribute (Column)	A named property of an entity
Domain	The set of permissible values for an attribute
Degree	Number of attributes in a relation
Cardinality	Number of tuples in a relation

2.2 Keys in Relational Model

Super Key: *Any set of attributes that uniquely identifies a tuple*

Candidate Key: *Minimal super key — no redundant attributes*

Primary Key: *The chosen candidate key used to uniquely identify each tuple*

Foreign Key: *An attribute referencing the primary key of another relation*

Composite Key: *A key formed by combining two or more attributes*

2.3 SQL — Structured Query Language

SQL is the standard language for interacting with relational databases. It is divided into sub-languages based on functionality:

Sub-language	Commands	Purpose
DDL — Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define and modify schema structure
DML — Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate data within tables
DCL — Data Control Language	GRANT, REVOKE	Manage user permissions
TCL — Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

2.4 SQL Query Examples

Creating a table and inserting data:

```
CREATE TABLE Students ( student_id INT PRIMARY KEY, name VARCHAR(100) NOT NULL,
department VARCHAR(50), cgpa FLOAT );
```

```
INSERT INTO Students VALUES (101, 'Ananya Roy', 'CSE', 9.1); INSERT INTO Students VALUES (102, 'Rahul Das', 'ECE', 8.4);
```

Querying with filters and ordering:

```
SELECT name, cgpa FROM Students WHERE department = 'CSE' ORDER BY cgpa DESC;
```

Joining two tables:

```
SELECT S.name, E.course_name FROM Students S JOIN Enrollments E ON S.student_id = E.student_id WHERE E.grade = 'A';
```

2.5 Summary

The relational model provides a mathematically sound basis for organizing data. SQL enables users to express complex queries in a declarative manner. Mastery of keys, joins, and SQL commands is essential before progressing to normalization and transaction management.

Introduction to Database Management Systems

Chapter 3 — Normalization

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. It involves decomposing tables into smaller, well-structured relations while preserving the original information and dependencies.

3.1 Anomalies in Unnormalized Tables

Anomaly Type	Description	Example
Insertion Anomaly	Cannot insert data without providing unrelated data	Cannot add a course unless a student enrolls in it
Deletion Anomaly	Deleting one fact causes loss of another	Deleting last student in a course removes course info
Update Anomaly	Changing one fact requires multiple row updates	Updating professor's department requires many chan

3.2 Functional Dependencies

A functional dependency X → Y means the value of attribute set X uniquely determines the value of attribute set Y in a relation.

Types of functional dependencies:

- Trivial FD: Y is a subset of X (e.g., {A, B} → A)
- Non-trivial FD: Y is not a subset of X (e.g., student_id → name)
- Partial Dependency: A non-key attribute depends on part of a composite key
- Transitive Dependency: A non-key attribute depends on another non-key attribute

3.3 Normal Forms

Normal Form	Condition	Removes
1NF — First Normal Form	All attributes are atomic; no repeating groups	Multi-valued attributes
2NF — Second Normal Form	In 1NF + no partial dependencies on a candidate key	Partial dependencies
3NF — Third Normal Form	In 2NF + no transitive dependencies	Transitive dependencies
BCNF — Boyce-Codd NF	For every FD X → Y, X must be a superkey	Anomalies not handled by 3NF
4NF — Fourth Normal Form	In BCNF + no multi-valued dependencies	Multi-valued dependencies

3.4 Step-by-Step Normalization Example

Consider an unnormalized table: StudentCourse(student_id, student_name, course_id, course_name, instructor, instructor_dept)

- 1NF: Already atomic — no repeating groups present
- 2NF: student_name depends only on student_id (partial dependency on composite key). Decompose into Students(student_id, student_name) and Enrollments(student_id, course_id, instructor)
- 3NF: instructor_dept depends on instructor (transitive dependency). Decompose into Instructors(instructor, instructor_dept) and update Enrollments accordingly

Key Insight: Every step of normalization trades redundancy for more tables. Always verify that decomposition is lossless (original table can be reconstructed via joins) and dependency-preserving.

3.5 Summary

Normalization is a systematic technique to produce a clean, efficient database schema. Understanding functional dependencies is the key to applying normal forms correctly. In practice, most production databases are normalized to 3NF or BCNF.